This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset LaTeX solutions.

1.a

Feature vector for "so so worried about tomorrow".

Taking the vocabulary [about, amazing, angry, day, hate, love, of, scared, so, spiders, this, tomorrow, waiting, worried], each entry counts how many times that word appears in the tweet, giving

$$f(x) = [\,1,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 2,\ 0,\ 0,\ 1,\ 0,\ 1\,].$$

Only four entries are non-zero: *about* in position 1, *so* in position 9, *tomorrow* in position 12 and *worried* in position 14. Position 9 holds 2 because "so" appears twice; every other vocabulary word is absent from this tweet and holds 0.

1.b

Softmax of $\mathbf{z} = [2.0, 1.0, -1.0]$.

Exponentiate each logit:

$$e^{2.0} = 7.389, \qquad e^{1.0} = 2.718, \qquad e^{-1.0} = 0.368.$$

Every class shares the same denominator, the sum of those three:

$$\sum_{j=1}^{3} e^{z_j} = 7.389 + 2.718 + 0.368 = 10.475.$$

Dividing each exponential by that sum gives

$$p = \left[\frac{7.389}{10.475},\ \frac{2.718}{10.475},\ \frac{0.368}{10.475}\right] = [\,0.705,\ 0.259,\ 0.035\,],$$

so $P(\text{Joy}) = 0.705$, $P(\text{Anger}) = 0.259$ and $P(\text{Fear}) = 0.035$.

1.c

Loss for "so angry", true label Anger.

Writing all three terms out, the one-hot label $\mathbf{y} = [0, 1, 0]$ zeroes two of them:

$$\text{Loss}_{\text{CE}} = -\big(0 \cdot \log 0.2 + 1 \cdot \log 0.7 + 0 \cdot \log 0.1\big) = -\log 0.7 = 0.357.$$

For on-hot labels, the loss reads the probability the model gave for the true class and ignores the other classes.

Behaviour of the loss. As the predicted probability for the correct class approaches 1, $\log 1 = 0$ so the loss approaches 0. As that probability approaches 0, the loss grows without bound, since the log of a number approaching 0 goes to negative infinity and the leading minus sign flips it to positive infinity. So the loss is bounded below by 0 at the good end but has no ceiling at the bad end, which is what makes a confident mistake expensive and punishing a confident bad answer.

1.d

(i) Derivation.

Substituting the softmax into the loss,

$$\text{Loss}_{\text{CE}} = -\sum_i y_i \log p_i = -\sum_i y_i \log\left(\frac{e^{z_i}}{\sum_j e^{z_j}}\right).$$

The log of a quotient splits into a difference of logs, and $\log e^{z_i} = z_i$:

$$\text{Loss}_{\text{CE}} = -\sum_i y_i z_i + \left(\sum_i y_i\right) \log \sum_j e^{z_j} = -\sum_i y_i z_i + \log \sum_j e^{z_j},$$

the bracket drops out because the one-hot entries sum to 1.

Now differentiate with respect to a single logit z_k, term by term. In the first sum because only when $i = k$ term contains z_k, and the remaining terms are constants with respect to z_k, so it survives as a coefficient:

$$\frac{\partial}{\partial z_k}\left(-\sum_i y_i z_i\right) = -y_k.$$

In the second term, similar to the first term, z_k appears in exactly one term of the denominator sum, so the chain rule gives

$$\frac{\partial}{\partial z_k} \log \sum_j e^{z_j} = \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k.$$

Both sums collapse to their single matching term, so no sum remains in the result.

(ii) Final expression.

$$\frac{\partial \text{Loss}_{\text{CE}}}{\partial z_k} = p_k - y_k.$$

Read case by case, the target class has $y_k = 1$ and so gets $p_k - 1$, while every other class has $y_k = 0$ and gets p_k.

(iii) Why this makes sense for learning. Our goal is to minimize gradient when it gets close to the target class. The gradient vanishes exactly when the prediction matches the label: the target class's probability approaches 1 so $p_k - 1$ approaches 0, and the other probabilities approach 0 so their gradients do too. The signs point the right way, since the target class's gradient is negative and a descent step therefore raises its logit, while every other class has a positive gradient and gets pushed down, hardest on whichever wrong class currently holds the most probability. The size of each entry is simply how far that class's predicted probability sits from its label, so a confident mistake produces a large correction and an almost correct prediction produces almost none.

2.a

Averaged embedding for "so angry".

The tweet has two words, with embeddings so $= [0.1, 0.0]$ and angry $= [-0.6, -0.8]$. Averaging position by position over the two words,

$$\left[\frac{0.1 + (-0.6)}{2},\ \frac{0.0 + (-0.8)}{2}\right] = [\,-0.25,\ -0.40\,].$$

The result stays 2 numbers wide, and it lands in the negative quadrant, which fits an angry tweet.

Benefit. The input size stays fixed as the vocabulary grows. Bag-of-words needs one more slot for every new word, so 14 words give 14 slots and a realistic 50,000-word vocabulary gives 50,000 mostly-zero slots, while the averaged embedding is 2 numbers wide whatever the vocabulary size. Similar words also sit near each other in the embedding space, so what the model learns about one word partly transfers to another, which two independent bag-of-words slots cannot do.

Limitation. Averaging washes out meaning when words pull in opposite directions, since the result comes out neutral. Using the given embeddings, a tweet containing both love $= [0.9, 0.4]$ and hate $= [-0.8, -0.5]$ averages to roughly $[0.05, -0.05]$, which is indistinguishable from a tweet of neutral words such as "of this". The same effect dilutes a strong word when the tweet also contains several words sitting at or near the origin, because the average gives every word equal weight regardless of how much meaning it carries.

2.b

Forward pass for "so angry".

Node/Variable	Formula/Computation	Value
$\mathbf{x}$	NA	$[-0.25,\ -0.40]$
$\mathbf{h}$	$h_1 = 1.0(-0.25) + 0.5(-0.40) + 0.1 = -0.35$ $h_2 = -0.5(-0.25) + 1.0(-0.40) - 0.2 = -0.475$ $\mathrm{ReLU}([-0.35,\ -0.475])$	$[0,\ 0]$
z	$\mathbf{W}^{(2)}\mathbf{h} + b^{(2)} = 0.8(0) + (-0.6)(0) + 0.3$	0.3
$\hat{y}$	$\sigma(z) = 1/(1 + e^{-0.3}) = 1/(1 + 0.741)$	0.574

2.c

Loss.

Variable	Formula/Computation	Value
L	$-[\,y \log \hat{y} + (1-y)\log(1-\hat{y})\,]$ $= -[\,0 \cdot \log 0.574 + 1 \cdot \log(1 - 0.574)\,] = -\log 0.426$	0.854

Gradients.

Gradient	Formula (Chain Rule)	Evaluated Value
$\frac{\partial L}{\partial \hat{y}}$	$-\left[y \cdot \frac{1}{\hat{y}} + (1-y) \cdot \frac{1}{1-\hat{y}} \cdot (-1)\right]$ $= \frac{1-y}{1-\hat{y}} - \frac{y}{\hat{y}}$ $= \frac{1-0}{1-0.574} - \frac{0}{0.574} = \frac{1}{0.426}$	2.347
$\frac{\partial L}{\partial z}$	$\frac{\partial L}{\partial \hat{y}} \cdot \hat{y}(1-\hat{y}) = 2.347 \times (0.574)(0.426)$	0.574
$\frac{\partial L}{\partial \mathbf{h}}$	$\frac{\partial L}{\partial z} \cdot \mathbf{W}^{(2)} = 0.574 \times [0.8,\ -0.6]$	$[0.459,\ -0.344]$
$\frac{\partial L}{\partial \mathbf{W}^{(2)}}$	$\frac{\partial L}{\partial z} \cdot \mathbf{h} = 0.574 \times [0,\ 0]$	$[0,\ 0]$
$\frac{\partial L}{\partial b^{(2)}}$	$\frac{\partial L}{\partial z} \cdot 1$	0.574
$\frac{\partial L}{\partial \mathbf{W}^{(1)}}$	$\frac{\partial L}{\partial \mathbf{h}} \odot \mathrm{ReLU}'([-0.35, -0.475])\ \mathbf{x}^\top$	$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$
$\frac{\partial L}{\partial \mathbf{b}^{(1)}}$	$\frac{\partial L}{\partial \mathbf{h}} \odot \mathrm{ReLU}'([-0.35, -0.475])$	$[0,\ 0]$

2.d

Spatial clustering. The positive emotion words sit in the upper right quadrant, with amazing $= [0.8, 0.6]$, and the negative emotion words sit in the lower left, with angry $= [-0.6, -0.8]$, and worried $= [-0.3, -0.6]$. The neutral function words are gathered at or beside the origin, since of and this are both exactly $[0.0, 0.0]$ and so and about are within 0.1 of it.

Distance from the origin marks the strength of the emotion rather than only its direction: amazing and angry both with distance of 1, while day at 0.22 and waiting at 0.32 carry the same signs but far weaker feeling. The space also reflects the company a word keeps rather than its dictionary meaning alone. Neither spiders nor waiting is inherently negative, yet both land in the negative region because they appear in negative contexts.

Semantic similarity and the advantage over bag-of-words. Because words used in similar ways end up near each other, the geometry itself carries meaning: what the model learns about one word transfers to its neighbours, so a word it has barely seen is still usable if it sits near words it knows. Bag-of-words cannot do this, since it only counts occurrences and gives every word its own independent slot, so amazing and love are as unrelated to each other as either is to hate, and learning about one teaches nothing about the others.

3.h

Three learning rates. All runs use the default 15 epochs on the bag-of-words linear classifier. For reference, a model predicting all three classes equally has a loss of $\log 3 = 1.099$, and always guessing the most common class gives an accuracy of 0.582.

Learning rate	Training loss	Validation accuracy	Final accuracy
0.02	$1.16 \to 1.08$, still falling	$0.351 \to 0.437$	0.4372
0.2	$1.16 \to 1.02$	$0.436 \to 0.554$	0.5544
2.0	$1.17 \to 4.26 \to 2.58$	$0.582 \to 0.500 \to 0.582$	0.5821

At 0.02 the loss falls steadily but never reaches $\log 3$ within the 15 epochs, so the model is still worse than uniform when training stops and the accuracy is the lowest of the three. At 0.2 the loss drops below the uniform landmark and the accuracy climbs to 0.554, the best of the three. At 2.0 the loss rises to well above uniform and the accuracy swings between epochs, ending back at exactly the majority baseline, which means the model has stopped distinguishing the classes at all.

Why. Each step moves the weights by the learning rate times the gradient. Too small a rate means the weights cannot travel far enough in the epochs available, so training ends while the loss is still falling. Too large a rate means each step overshoots the minimum and lands further up the other side, so the loss grows instead of shrinking and the accuracy swings rather than settling. There is a threshold set by the curvature of the loss: below it the distance to the minimum shrinks each step, and above it the iteration diverges.

One further observation. The scale of the initial weights mattered more than the learning rate did. The handout initialises with `np.random.randn` over 11757 features, which makes the starting scores large and the softmax saturated. Across six random seeds at the default settings that gives an average accuracy of 0.414, ranging from 0.361 to 0.482 and falling short of the required 0.45 on four of the six. Scaling the same random initialisation by 0.1 inside my own code region raises it to 0.554, because each weight then starts near where it needs to be rather than a full unit away, and the averaged gradient for any single word is far too small to cover that distance in 15 epochs. The handout initialisation line, the learning rate and the epoch count are all unchanged.

5.a
5.b
5.c
5.d
5.e